

Nonparametric SLR - Examples in R

B.A. Hartlaub and A.S. Wagaman

Examples for Nonparametric Simple Linear Regression (SLR)

Example 1 - Toy Example (6 pairs of data points)

This is a toy example on only 6 pairs of observations for two variables to help demonstrate the process employed. Be sure you understand what is being done along the way.

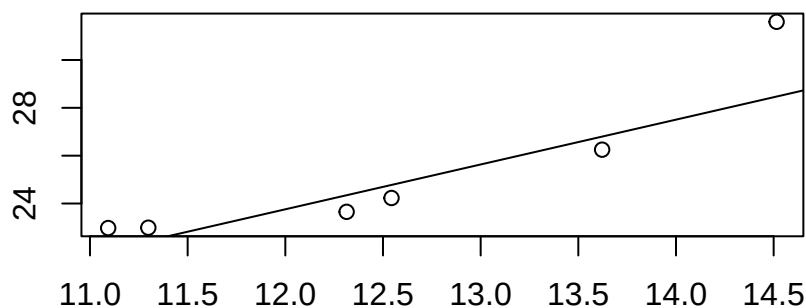
The data was generated from a model where the true slope is 2, so we can see if we recover the slope, despite an n of 6.

```
# Data
x <- c(12.54367, 11.09313, 13.62189, 12.31389, 14.51518, 11.29860)
y <- c(24.23092, 22.97545, 26.25129, 23.65294, 31.59527, 22.98974)
data <- cbind(x,y)
```

The function described in the text (Hollander, Wolfe, and Chicken Chapter 9) to fit the model is the *theil* function.

Let's try fitting the model with the data in its current format, with all the default options.

```
theil(x, y, beta.0 = 0, type = "t")
```



```
## Alternative: beta not equal to 0
## C = 15, C.bar = 1, P = 0.003
## beta.hat = 1.874
## alpha.hat = 1.272
##
## 1 - alpha = 0.95 two-sided CI for beta:
```

```
## 0.555, 3.735
```

Besides the output with estimates, the function plots the data points and fitted line (not as nicely as `ggformula` or `ggplot2` graphics).

You can get the plot to not print by setting the `doplot` argument to `FALSE`. You can also get the slopes generated to be listed out by setting `slopes = TRUE`. Remember there are $n(n - 1)/2$ slopes, so the list is VERY long for even moderate sample sizes. We demo these below. The final slope estimate is the *median* of all these possible slopes.

```
theil(x, y, beta.0 = 0, type = "t", doplot = FALSE, slopes = TRUE)
```

```
## Alternative: beta not equal to 0
## C = 15, C.bar = 1, P = 0.003
## beta.hat = 1.874
## alpha.hat = 1.272
##
## All slopes:
## i j      S.ij
## 1 2 0.06954787
## 1 3 0.55497395
## 1 4 0.86551905
## 1 5 1.29543333
## 1 6 2.51890533
## 2 3 0.65321238
## 2 4 0.99687568
## 2 5 1.40384971
## 2 6 2.67536638
## 3 4 2.51536252
## 3 5 1.98650612
## 3 6 3.60803438
## 4 5 1.87380127
## 4 6 3.73538557
## 5 6 5.98235735
##
##
## 1 - alpha = 0.95 two-sided CI for beta:
## 0.555, 3.735
```

We can grab the slopes to verify that the estimate is the median by saving the `theil` output object.

In the names saved for the object, we can see the `slopes.table` is there. It has 3 columns, with the slopes in the final column. We can grab the slopes to view, or save them to another object, and then compute the median.

```
toutput <- theil(x, y, beta.0 = 0, type = "t", doplot = FALSE, slopes = TRUE)
names(toutput)
```

```
## [1] "stat.name"      "type"           "alpha"          "slopes"         "r"
## [6] "beta.0"         "p.val"          "slopes.table"   "C.stat"         "C.bar"
## [11] "alpha.hat"      "beta.hat"       "L"              "U"
```

```
toutput$slopes.table[, 3] # just pull slopes
```

```
##
## 0.06954787 0.55497395 0.86551905 1.29543333 2.51890533 0.65321238 0.99687568
##
## 1.40384971 2.67536638 2.51536252 1.98650612 3.60803438 1.87380127 3.73538557
##
```

```
## 5.98235735
```

```
slopesvec <- toutput$slopes.table[, 3] # save slopes to a vector  
median(~ slopesvec) # or use faustats()
```

```
## [1] 1.873801
```

We can verify the p-value manually using the value of $C.bar$ in the output, using symmetry.

```
2*pKendall(-1, N = 6)
```

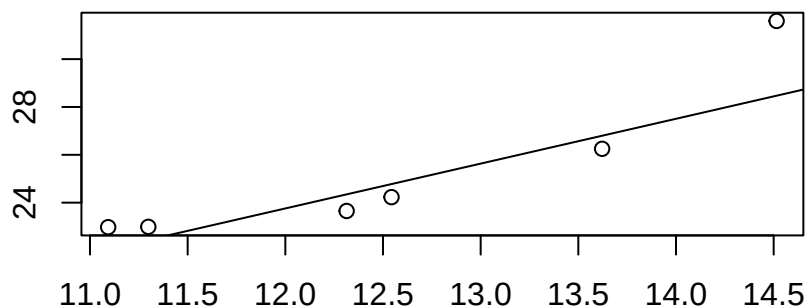
```
## [1] 0.002777778
```

We can see the p-value of 0.003 reported was slightly rounded. There is actually an option for this in the *theil* function, *r*, which has a default of 3. So if you are getting p-values of 0 or 1, you can increase *r* to get more digits.

Type = “t” means two-sided. The other options are “l” and “u” for lower and upper.

By default, the test for slope was for a two-sided test to check if the slope is different from 0. Here, that test showed significance, and we know the true slope from the underlying model is 2. So, let’s see if we detect a difference in slope from a value of 2.5 (slightly off, but in the ballpark). We demo increasing *r* slightly as well so you can see the rounding change.

```
# check if slope is different from 2.5, change beta.0 value  
theil(x, y, beta.0 = 2.5, type = "t", r = 6) # r is number of places for rounding
```



```
## Alternative: beta not equal to 2.5
```

```
## C = -3, C.bar = -0.2, P = 0.719444
```

```
## beta.hat = 1.873801
```

```
## alpha.hat = 1.272492
```

```
##
```

```
## 1 - alpha = 0.95 two-sided CI for beta:
```

```
## 0.554974, 3.735386
```

```
2*pKendall(-0.2, N=6) # can manually check p-value
```

```
## [1] 0.7194444
```

How would you manually check that $C(K)$ is -3? You can look for concordance, assuming you generate the appropriate values. We need to get $y - 2.5x$, but we need to do this when the *x*’s are in *sorted* order from

smallest to largest.

```
mydata <- data.frame(x, y)
mydata2 <- arrange(mydata, x)
# if you wanted to compute D's to check that C = -3
mydata2 <- mutate(mydata2, D = y - 2.5*x)
mydata2$D
```

```
## [1] -4.757375 -5.256760 -7.131785 -7.128255 -7.803435 -4.692680
```

We see only 6 concordant pairs (verify yourself!), so the rest are discordant. $n = 6$, so the total number of pairs is $6(5)/2 = 15$. Thus 9 pairs are discordant (as long as there are no ties, the number of concordant and discordant pairs summed = total number of pairs).

So, $C = K = 6 - 9 = -3$, confirming the output.

Now we turn to validating the confidence interval construction. Remember that finding the CI is a multi-step process. Finding the critical value is the first step to finding the appropriate positions in the sorted list of slopes to create our interval. The critical value is used to compute C_α , which is then used to compute M and $Q + 1$, the values used to indicate the positions of the ordered slopes for the endpoints of the interval. (This notation is what is used in Hollander, Wolfe, and Chicken.) So we start with getting the critical value.

```
# CI construction validation
qKendall(0.025, N = 6)
```

```
## [1] -0.7333333
```

```
pKendall(-0.733333, N = 6) # less than 95% because this is over 0.025
```

```
## [1] 0.02777778
```

```
qKendall(c(0.01, 0.025, by = 0.005), N = 6)
```

```
## [1] -0.7333333 -0.7333333 -0.8666667
```

```
pKendall(-0.86667, N = 6) # appears to be next lowest
```

```
## [1] 0.008333333
```

```
1-2*pKendall(-0.86667, N = 6) # generates 98.33% CI
```

```
## [1] 0.9833333
```

This gives us the negative critical value. We make it positive (via symmetry), and use it to construct C_α . The C_α is used to get M and $Q + 1$ to pull out the desired slopes.

```
n <- 6
N <- n*(n-1)/2 # number of pairs
# rounded because of decimals, was just over 11, now 11
Calpha <- round(n*(n-1)/2*(0.8667)-2,0); Calpha
```

```
## [1] 11
```

```
M <- (N - Calpha)/2; M
```

```
## [1] 2
```

```
Q <- M + Calpha; Q+1
```

```
## [1] 14
```

So, $M = 2$ and $Q + 1 = 14$. This indicates we should take the 2nd and 14th slopes (out of the 1-15 possible sorted slopes - so 1 in from each side). This matches the theil output above!

The CI from theil should be at LEAST what you request in terms of coverage, so it matches here. Here, it does go over 95% confidence (the idea is not to go under).

Finally, we look at the intercept estimator to verify what we see in the output above. The idea is to construct the estimated intercepts from the data with the estimated slope used for the line. Then, their median is taken as the estimated intercept.

```
# Intercept estimation check
A <- y - 1.874*x # generates all A values
sort(A)

## [1] 0.5767101 0.7238681 0.7240824 1.8161636 2.1869244 4.3938227

median(A) # their median is the estimate of alpha

## [1] 1.270123
```

The minor difference here looks to be due to rounding, as the output had 1.272492 (when we used 6 digits), slightly different from this 1.270123. Using what is in the *theil* output is fine.

Workplace Mileage Example

Story: Do flexible work schedules reduce the demand for resources? The Lake County, Illinois, Health Department experimented with a flexible four-day workweek. For a year, the department recorded the mileage (and other variables) driven by 11 field workers on an ordinary five-day workweek. Then it changed to a flexible four-day workweek and recorded mileage (and other variables) for a year for the same workers in that setting.

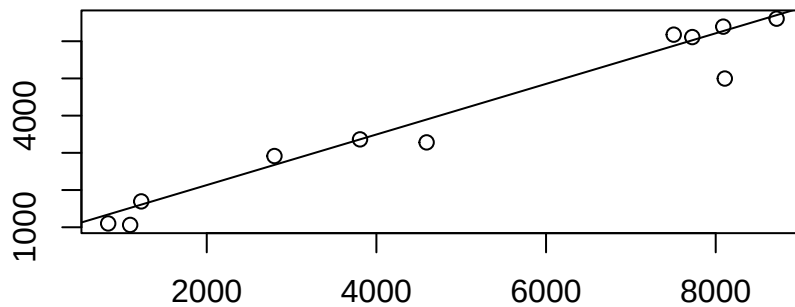
Previously, we explored whether or not the four-day workweek reduced mileage. We could just study the relationship between the five-day and four-day workweek mileages though, using regression. There are $n = 11$ pairs of observations.

```
# Data
day5 <- c(2798, 7724, 7505, 838, 4592, 8107, 1228, 8718, 1097, 8089, 3807) # use as X
day4 <- c(2914, 6112, 6177, 1102, 3281, 4997, 1695, 6606, 1063, 6392, 3362)

# cleaner to sort if data is in a dataframe, if you want to be able to check concordance
mileagedata <- data.frame(day5, day4)
# arrange will put the obs in order of the day5 values,
# keeping their day4 values with them
mileagedata2 <- arrange(mileagedata, day5)
mileagedata2 # to view data

##   day5 day4
## 1   838 1102
## 2  1097 1063
## 3  1228 1695
## 4  2798 2914
## 5  3807 3362
## 6  4592 3281
## 7  7505 6177
## 8  7724 6112
## 9  8089 6392
## 10 8107 4997
## 11 8718 6606

theil(day5, day4, beta.0 = 0, type = "t", r = 6) # test for a significant relationship
```



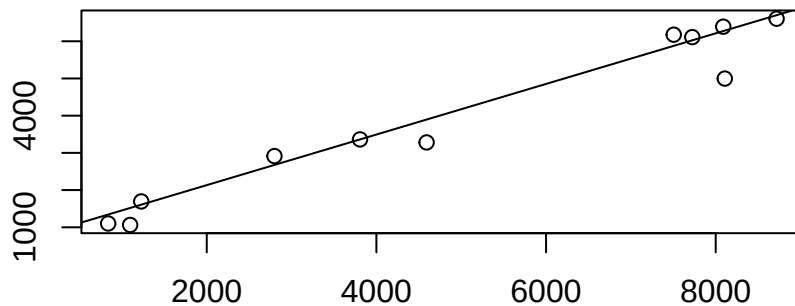
```
## Alternative: beta not equal to 0
## C = 43, C.bar = 0.781818, P = 0.000333
## beta.hat = 0.679957
## alpha.hat = 773.404095
##
## 1 - alpha = 0.95 two-sided CI for beta:
## 0.471463, 0.761883
```

```
2*pKendall(-0.781818, N = 11) # manually compute p-value, using symmetry
```

```
## [1] 0.0003334436
```

We set $r=6$ here because if you use $r=3$, the p-value is rounded to 0 in the *theil* output. With $r = 6$, we still see it is small, at 0.000333, but doesn't show up as 0.

```
# can also run when x and y are in a dataset
with(mileagedata2, theil(day5, day4, beta.0 = 0, type = "t", r = 6))
```



```
## Alternative: beta not equal to 0
```

```
## C = 43, C.bar = 0.781818, P = 0.000333
## beta.hat = 0.679957
## alpha.hat = 773.404095
##
## 1 - alpha = 0.95 two-sided CI for beta:
## 0.471463, 0.761883

# what would testing beta = 1 or not tell you?
with(mileagedata2, theil(day5, day4, beta.0 = 1, type = "t", r = 6))

## Alternative: beta not equal to 1
## C = -47, C.bar = -0.854545, P = 4.6e-05
## beta.hat = 0.679957
## alpha.hat = 773.404095
##
## 1 - alpha = 0.95 two-sided CI for beta:
## 0.471463, 0.761883
```

Remember you could turn on the potential slopes to see them if desired. (Remember the list gets longer and longer for larger n though.)

Data Sources

Simulated data for the Toy example (6 data points).

Charles S. Catlin, "Four-day Work Week Improves Environment," Journal of Environmental Health, Denver, 59:7, 1997.

Standard Reference

These materials were created for use in an undergraduate nonparametrics course. Where provided, textbook references refer to Nonparametric Statistical Methods (3rd Edition) by Hollander, Wolfe, and Chicken. Notation is chosen to be consistent with that text. However, materials may be used independently of the textbook.

License

This work is protected by creative commons license CC BY-NC-SA.